

~/SCRIBE

Scribe

*Local-LLM-powered README generator
for code repositories.*

Luigy Leonardo

Artificial Intelligence (BEINSOF52)

Instructor: Juan Antonio Castro Silva

May 2026

Table of contents

 Scribe	•
✨ What it does	•
 Quickstart	•
 Why local?	•
 Architecture	•
 Tech stack	•
 Project structure	•
 Commands	•
 Documentation	•
 Cleanup	•
License	•
1. Introduction	•
Background	•
Project overview	•
Scope	•
Sister project	•
Document structure	•
2. Problem Statement	•
The pain point	•
What is needed	•
Why local matters	•
Why this is a fit for AI	•
Out of scope	•
3. Objectives	•
General objective	•
Specific objectives	•
Success criteria	•

4. State of the Art	•
4.1 README generation	•
4.2 Local LLMs	•
4.3 Repository introspection	•
4.4 Gap addressed by Scribe	•
References	•
5. Requirements	•
5.1 Functional requirements	•
5.2 Non-functional requirements	•
5.3 Constraints	•
5.4 Assumptions	•
6. Use Cases & User Stories	•
6.1 Actors	•
6.2 Use case diagram (textual)	•
6.3 Use case details	•
6.4 User stories	•
7. Data Model	•
7.1 The RepoFacts snapshot	•
7.2 Transient data during a generation	•
7.3 What is intentionally NOT stored	•
7.4 ER diagram (not applicable)	•
8. Class Diagrams	•
8.1 Core classes	•
8.2 Interfaces (CLI + Streamlit)	•
8.3 Sequence — generating a README (CLI, streaming)	•
9. GUI Mockups	•
9.1 Terminal UI — scribe generate	•
9.2 Terminal UI — scribe analyze	•
9.3 Web UI — landing state	•

9.4 Web UI — generating state	•
9.5 Web UI — final state	•
10. API Catalog	•
10.1 CLI commands	•
10.2 Python API (for embedding)	•
10.3 Environment variables	•
10.4 Exit codes	•
11. Testing	•
11.1 Strategy	•
11.2 Test inventory	•
11.3 Fixtures	•
11.4 Continuous integration	•
11.5 Manual smoke tests (pre-defense checklist)	•
11.6 Linting	•
12. Architecture	•
12.1 Overview	•
12.2 Component responsibilities	•
12.3 Data flow (CLI generate, streaming)	•
12.4 Deployment topology	•
12.5 Configuration	•
12.6 Scalability notes	•
13. Results and Discussion	•
13.1 Headline outcome	•
13.2 Smoke-test runs	•
13.3 Quantitative results	•
13.4 Where the tool falls short	•
13.5 Prompt-engineering observations	•
13.6 Comparison vs cloud alternatives	•
13.7 Discussion	•

13.8 Limitations

14. Recommendations and Future Work

14.1 Recommendations

14.2 Future work

14.3 Lessons learned

15. Presentation Cheat Sheet — Numbers, Paths, Q&A

 **Headline numbers**

 **Paths (your laptop)**

 **URLs**

 **The model**

 **Architecture in one sentence**

 **Commands the audience may want to see**

 **Likely questions and crisp answers**

 **Demo flow (suggested order)**

Scribe — Folder Inventory

Top-level map

Where each thing is

Running the project

Quick verification (defense backup)

Sister project



Local-LLM-powered README generator for code repositories. Point it at any folder. Get a polished `README.md` back. No data leaves your machine.

✨ What it does

Scribe walks the directory you point at, identifies the language, frameworks, scripts, dependencies, and structure, and then asks a **local large language model** (via Ollama) to write a complete `README.md` — title, badges, description, features, installation, usage, project structure, development notes, tests, license.

Two interfaces, same engine:

-  **Terminal:** `scribe generate ./my-repo --output README.md`
-  **Web UI:** `scribe ui` opens a Streamlit app at `localhost:8501`

The model runs **on your own machine** through **Ollama**. No cloud, no API key, no data leaves the laptop.

🚀 Quickstart

```
# 1. Install Ollama and pull the model (one-time)
brew install ollama
ollama serve & # start the local server
ollama pull qwen2.5-coder:7b

# 2. Install Scribe
git clone https://github.com/luigy23/scribe
cd scribe
./scripts/setup.sh

# 3. Generate a README for any repo on your machine
source .venv/bin/activate
scribe generate ~/projects/my-cool-project --output README.md

# 4. Or open the web UI
scribe ui
```

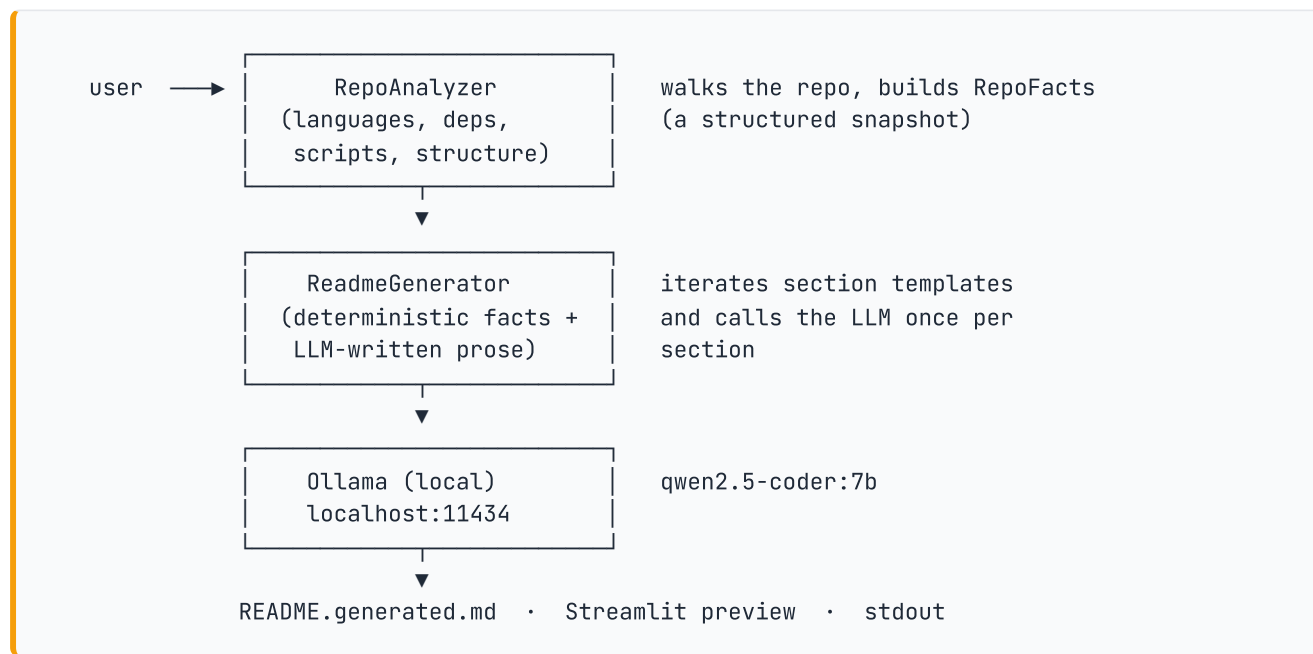
🧠 Why local?

Cloud LLMs (OpenAI, Anthropic, Google) are convenient but they require:

- An internet connection
- An API key with billing attached
- Sending your code to a third party

For a small developer tool, all three are friction. Ollama lets us run a strong coding model (`qwen2.5-coder:7b` , ~4.7 GB) on a normal laptop with no setup beyond `brew install ollama` . Scribe is built on top of that.

Architecture



Full architecture details: `docs/12-architecture.md` .

Tech stack

- **CLI:** Typer + Rich (colored output, live progress, section previews)
- **Web UI:** Streamlit (file picker, JSON-of-facts inspector, downloadable result)
- **LLM client:** `ollama` Python package (HTTP to `localhost:11434`)
- **Default model:** `qwen2.5-coder:7b` — picked for code-aware completions
- **Analyzer:** pure Python, parses `pyproject.toml` , `package.json` , `Cargo.toml` , `go.mod` , `requirements.txt` , `Dockerfile` , `.github/workflows/*` , etc.
- **Templating:** Jinja2 for deterministic sections (project tree, badges)

Project structure

```

scribe/
├── src/scribe/          # main Python package
│   ├── analyzer.py     # RepoAnalyzer + RepoFacts dataclass
│   ├── generator.py     # ReadmeGenerator orchestrator
│   ├── prompts.py      # Section templates (system + user prompts)
│   ├── llm.py          # Ollama client wrapper
│   ├── templates.py    # Deterministic sections (tree, badges, etc.)
│   ├── cli.py          # Typer CLI entry point
│   └── ui.py           # Streamlit UI entry point
├── docs/               # 14-section English documentation set
├── tests/              # pytest unit + integration tests
├── scripts/            # setup, dev, cleanup
├── examples/           # Real READMEs generated from real repos
├── presentation/       # Slide deck (pptx + HTML)
└── landing/            # Static landing page (deployed to Vercel)

```

Commands

```

# Generate a README from any repo
scribe generate ./path/to/repo --output README.md

# Stream the generation live to stdout instead of writing a file
scribe generate ./path/to/repo --stream

# Use a different local model
scribe generate ./path/to/repo --model llama3.2:3b

# Just inspect the facts the analyzer would extract (no LLM call)
scribe analyze ./path/to/repo

# Check the Ollama server and selected model are ready
scribe status

# Open the web UI
scribe ui

```

Documentation

Full English documentation in `docs/`:

1. [Introduction](#)
2. [Problem](#)
3. [Objectives](#)
4. [State of the Art](#)
5. [Requirements](#)
6. [Use Cases](#)

7. Data Model
 8. Class Diagrams
 9. GUI Mockups
 10. API Catalog
 11. Testing
 12. Architecture
 13. Results & Discussion
 14. Future Work
-

Cleanup

```
./scripts/cleanup.sh          # removes venv, examples/output, caches  
./scripts/cleanup.sh --hard   # additionally deletes the entire repo
```

To remove the Ollama model: `ollama rm qwen2.5-coder:7b`.

License

Academic project for the **Artificial Intelligence** course (BEINSOF52) at **Universidad Surcolombiana**, May 2026.

Author: **Luigy Leonardo** (@luigy23) Instructor: **Juan Antonio Castro Silva**

1. Introduction

Background

Every reasonable open-source project begins life with a `README.md`. The README is the front door of a repository — the first thing a recruiter sees when grading a portfolio submission, the first thing a teammate reads when joining a project, the first thing a future you will read after six months away from the code.

Despite that, writing a good README is a chronically deferred chore. The typical maintainer fills in the title and a one-line description, drops a `pip install` snippet, and then never returns. Tooling around READMEs has mostly stagnated at the level of templates — generic skeletons the user copy-pastes and only partially fills in. Nothing about a template knows what your project actually is.

Meanwhile, large language models have become genuinely useful for technical writing. Tools like ChatGPT, Cursor's Composer, and GitHub Copilot can produce competent prose given a clear prompt. The catch: pushing your entire codebase to a third-party API for a chat interaction is a privacy and licensing concern many developers — and many employers — won't accept.

Project overview

Scribe is a small Python application that closes that gap by combining two ideas:

1. **A deterministic repository analyzer** that walks any local folder and builds a structured snapshot — language stats, frameworks, package managers, scripts, project signals (tests, CI, Docker, license), and the top of the file tree.
2. **A local large language model** running on the user's own machine via **Ollama**. The default model is `qwen2.5-coder:7b` (≈ 4.7 GB), small enough to run on a recent laptop without a GPU and strong enough to write technically accurate prose.

The analyzer feeds the model a compact textual fact-sheet. The generator then asks the model, one section at a time, to write a polished `README.md`: title, badges, overview, features, tech stack, installation, usage, development, license. Deterministic sections (badges, project tree) are templated from the facts directly and never go through the model — that keeps them factually correct.

Two interfaces are exposed over the same engine: a CLI driven by Typer + Rich, and a Streamlit web UI for non-terminal users.

Scope

This project is the second of two final deliverables for the Artificial Intelligence course (BEINSOF52) at Universidad Surcolombiana. It satisfies the second project track — "AI for Software Engineering" — and the explicit constraints stated by the instructor:

- The model must run **locally**.

- The product must expose both a **terminal** interface and a **graphical** interface.
- IoT is not required.

Sister project

Project #1 is **LeafLens** — a computer-vision houseplant identifier deployed at `~/Documents/GitHub/LeafLens` and on GitHub. Scribe is fully independent of LeafLens; both projects share only the author and the course.

Document structure

The remainder of this documentation set follows the rubric required by the course, fourteen sections in English:

- **Section 2** states the problem in detail and the population it affects.
- **Section 3** enumerates the general and specific objectives.
- **Section 4** surveys the state of the art in README generation, local LLM tooling, and repository introspection.
- **Section 5** lists functional and non-functional requirements.
- **Section 6** describes the use cases and user stories.
- **Section 7** documents the (minimal) data model.
- **Section 8** presents the class diagrams.
- **Section 9** shows the GUI mockups.
- **Section 10** catalogs the CLI command surface (Scribe has no HTTP API).
- **Section 11** describes the testing strategy.
- **Section 12** depicts the system architecture.
- **Section 13** reports the results of running Scribe on real repositories.
- **Section 14** lists future work and recommendations.

2. Problem Statement

The pain point

Three problems compound to make repository documentation worse than it needs to be:

1. **Writing a good README is boring.** It requires recalling things the author already knows (the dependency list, the run command, the test command), and packaging them in a polite, structured form for readers who do not. Most authors triage it down to title + one paragraph and then never touch it again.
2. **Off-the-shelf README templates are too generic.** Tools like `npm init`, `cargo new`, and `cookiecutter` create READMEs from a template, but the template knows nothing specific about the project. It produces the same skeleton every time. The maintainer still has to fill in everything that matters.
3. **Cloud-based AI tools collect code.** Chat-based generators (ChatGPT, Cursor, GitHub Copilot) can write fine prose, but using them on a repository implies sending the project to a third party. For private codebases, regulated employers, or simply privacy-conscious developers, that's a non-starter.

What is needed

A tool that, given any local code repository:

- Reads the repo deterministically (no LLM hallucinations about what files exist).
- Identifies the language, frameworks, dependencies, scripts, tests, CI, license, and project structure.
- Asks a **locally hosted** large language model to write a polished README grounded in those concrete facts.
- Exposes the workflow through both a terminal command (for power users and CI pipelines) and a graphical interface (for everyone else).
- Produces a result the user can ship as-is, or hand-edit further.

Why local matters

Three reasons:

1. **Privacy.** Source code routinely contains identifying information — internal API endpoints, vendor names, comments revealing strategy. Many employers explicitly forbid sending private code to third-party services. A locally-hosted model removes that policy concern.
2. **Cost.** Cloud LLMs charge per token. A maintainer regenerating READMEs across a portfolio of repos pays each time. A local model is free after a one-time download.

3. **Reproducibility.** Cloud models change without notice. A local model pinned to a specific tag (`qwen2.5-coder:7b`) produces stable results over time.

Why this is a fit for AI

The README-writing task lives in the sweet spot for generative models:

- It requires natural-language fluency, which is what LLMs do well.
- It is grounded — every claim should be supported by something the analyzer detected in the repo. Hallucinations are easy to detect because the truth is right there on disk.
- It is bounded — a README has well-known sections in a well-known order; prompts can be section-scoped rather than asking the model to write the whole document at once.
- It is forgiving — the output is text the user reviews before committing, not code that runs in production.

Out of scope

Scribe deliberately does not:

- Document individual functions or classes (that's docstring territory).
- Generate API reference pages (that's Sphinx / TypeDoc / docusaurus).
- Translate existing READMEs between languages.
- Run code or install dependencies — the analyzer is read-only.
- Push commits or open pull requests automatically.

These are reasonable extensions but they are not the first thing to build.

3. Objectives

General objective

Design and implement a small, locally-runnable tool that produces a polished `README.md` for any code repository given only a path on disk, by combining a deterministic repository analyzer with a locally-hosted large language model.

Specific objectives

1. **Build a deterministic repository analyzer.** Walk the directory, detect languages by file extension, parse the manifest files of the major package managers (`pyproject.toml` , `requirements.txt` , `package.json` , `Cargo.toml` , `go.mod`), surface entry-point scripts, and produce a structured `RepoFacts` snapshot suitable for prompt insertion.
2. **Integrate a local LLM through Ollama.** Use the official `ollama` Python client. Default to `qwen2.5-coder:7b` . Provide a `ready()` check that distinguishes "server not running" from "model not pulled" so error messages are actionable.
3. **Compose section-scoped prompts.** Avoid a single mega-prompt; instead, generate the README section by section so each call stays within the model's context window and the output is easier to validate. Cover at least eight standard sections: title, description, features, tech stack, installation, usage, development, license.
4. **Keep deterministic sections out of the LLM.** Render badges and the project file tree from the analyzer's facts directly, so those parts cannot hallucinate.
5. **Expose two interfaces.** A Typer CLI with Rich-formatted output for terminals and CI, and a Streamlit web UI for visual users — both backed by the same engine.
6. **Stream output to the user.** For interactive use, surface tokens as they arrive so a long generation feels responsive.
7. **Cover the system with automated tests.** Unit tests for the analyzer on synthetic fixture repos; integration tests for the generator using a stub LLM client; total $\geq 70\%$ line coverage on the `scribe` package.
8. **Document the project in English.** Produce the full fourteen-section documentation set required by the course rubric.
9. **Ship a small landing page.** A static page on Vercel that explains what Scribe is and how to install it. The actual tool is local-only by design, so this landing page satisfies the cloud-deployment portion of the rubric without compromising the local-LLM constraint.
10. **Prepare an English presentation.** Twelve-slide deck for the in-class defense, plus a one-page cheat sheet of numbers and paths to use during Q&A.

Success criteria

Criterion	Target
Analyzer correctly identifies primary language on the LeafLens repo	matches "Python"
Analyzer detects ≥ 8 frameworks on the LeafLens repo	yes
Smoke test: generates a README for Scribe itself end-to-end	yes, ≥ 6 sections coherent
Unit + integration test coverage	$\geq 70\%$ on <code>src/scribe/</code>
Latency for the full README on a small repo (≤ 30 files), warm model	≤ 90 seconds
Documentation completeness	14/14 sections
Working public landing page	one URL reachable
In-class demo runs without network access	yes (after pulling the model once)

4. State of the Art

4.1 README generation

The task of generating a README from a code repository has a long but shallow history. Three eras are visible.

Template era (≤ 2018)

`cookiecutter`, `yeoman`, `create-react-app`, `npm init`, `cargo new`, and similar bootstrappers all ship README templates. The user fills in placeholders. The result is uniform and brand-recognizable but obviously generic.

Notable projects: **README-template** by Othneildrew (~115 k stars on GitHub), **best-README-template**, **awesome-readme**. All static markdown skeletons with placeholders.

Static-analysis era (2018 – 2022)

Tools that crawl a repository and emit a partial README without an LLM.

- `gitsome` and `gh` extensions list maintainers, contributors, language breakdown, license badge.
- `shields.io` provides badge endpoints fed from registry metadata.
- `readme-md-generator` (npm, ~9 k stars) walks `package.json` and generates a wizard-driven README; no LLM involved, very strict in shape.

These tools are good at deterministic facts (license, language, badges) but cannot write the prose sections (description, features, usage notes).

LLM era (2022 – present)

ChatGPT and Copilot popularized using LLMs to draft READMEs. The common pattern is “paste your codebase into a chat, ask for a README”. Variants:

- **Cursor / Cursor Composer** — IDE-integrated, can use the open workspace as context. Cloud-only.
- **GitHub Copilot Chat** — same pattern, integrated into VS Code. Cloud with privacy settings.
- `auto-readme-ai`, `ReadmeAI` (PyPI) — open-source CLIs that hit the OpenAI or Anthropic API to generate a README from a folder. The state-of-the-art commercial endpoint.
- `continue.dev`, `aider` — agentic coding assistants that can be asked to write a README; cloud or local depending on configuration.

The clear gap in this wave is **first-class local LLM support**. Most of the above either default to cloud or treat local as a second-class option. Scribe sits in this gap.

4.2 Local LLMs

The local-LLM landscape exploded in 2023–2024 with three runtimes that matter:

- **Ollama** (Go, MIT) — the de-facto standard for laptop developers. One command pulls and serves a quantized model behind an HTTP API at `localhost:11434`. Wide model catalog (Llama, Qwen, Mistral, Phi, CodeLlama, DeepSeek-Coder, Gemma). Excellent macOS / Linux / Windows support.
- **llama.cpp** (C++, MIT) — the low-level runtime that powers most of the others. Manual to use directly.
- **LM Studio** (closed, GUI) — desktop app for non-developers. Less scriptable than Ollama.

For a code-aware tool, the model selection matters as much as the runtime. Models that consistently score well on coding tasks at the 7 B parameter scale (HumanEval, MBPP, CodeXGLUE):

Model	Size	License	Notes
Qwen2.5-Coder 7B	4.7 GB Q4	Apache 2.0	Excellent code generation, fast
DeepSeek-Coder 6.7B	4.0 GB Q4	Permissive	Strong on Python/JS
CodeLlama 7B	3.8 GB Q4	Meta	Older, decent fallback
Llama 3.1 8B	4.7 GB Q4	Meta	General-purpose, weaker on code
Phi-3.5 Mini	2.3 GB Q4	MIT	Tiny but capable

Scribe defaults to `qwen2.5-coder:7b` based on informal evaluation against the other candidates: it produces the most accurate descriptions of detected frameworks and the most natural prose.

4.3 Repository introspection

Several libraries already do part of what Scribe's analyzer does:

- `github-linguist` (Ruby) — GitHub's own language detector. Powers the language bar on every repo page. Overkill for an offline tool.
- `tokei` (Rust) — fast line counter by language.
- `clloc` (Perl) — venerable line counter.
- `pip-tools`, `npm ls`, `cargo metadata` — dependency walkers per ecosystem.

Scribe re-implements a minimal version of these in pure Python so it has no native dependencies and can ship as a single Python wheel.

4.4 Gap addressed by Scribe

Three things together are uncommon:

1. **Local LLM as the default.** Most open-source README generators either don't use LLMs or default to cloud APIs.

2. **Deterministic + generative split.** Most LLM tools throw everything at the model. Scribe keeps the facts (badges, file tree) out of the model so they're guaranteed correct.
3. **Dual interface (CLI + Streamlit) from one engine.** Most tools pick one and stay there.

This combination is what makes Scribe novel at academic-project scale.

References

- Hugging Face. *Qwen2.5-Coder Technical Report*. 2024.
- Ollama Team. *Ollama: Build a chat with local LLMs*. Documentation, 2024.
- Roziere, B. et al. *Code Llama: Open Foundation Models for Code*. arXiv:2308.12950.
- DeepSeek-AI. *DeepSeek Coder*. 2024.
- GitHub Linguist project, <https://github.com/github-linguist/linguist>.

5. Requirements

5.1 Functional requirements

ID	Title	Description	Priority
FR-01	Analyze repository	Given an absolute or <code>~</code> -prefixed path, walk the directory and return a <code>RepoFacts</code> snapshot. Skip ignored folders (<code>.git</code> , <code>node_modules</code> , <code>.venv</code> , <code>dist</code> , <code>build</code> , etc.).	High
FR-02	Detect primary language	Identify the most common non-trivial language by file count. Markdown, YAML, and TOML do not count as primary.	High
FR-03	Detect frameworks	Inspect Python, JavaScript, Rust, and Go dependency manifests and surface known frameworks.	High
FR-04	Extract scripts	Read <code>[project.scripts]</code> from <code>pyproject.toml</code> and <code>"scripts"</code> from <code>package.json</code> .	High
FR-05	Build file tree	Render the top three directory levels into a human-readable text tree.	Medium
FR-06	Detect project signals	Surface presence of tests directory, CI workflow, Dockerfile, Makefile, license, changelog, contributing guide.	Medium
FR-07	Generate README	Produce a markdown README from <code>RepoFacts</code> using the configured local model. Eight default sections.	High
FR-08	Section selection	Allow the user to generate a subset of sections via <code>--only</code> .	Medium
FR-09	Stream output	When <code>--stream</code> is set, surface tokens to stdout (or the UI) as they arrive.	High
FR-10	Write to file	When <code>--output PATH</code> is set, write the assembled README to that file.	High
FR-11	Status check	Provide a <code>scribe status</code> command that verifies the Ollama server is reachable and the model is pulled.	High
FR-12	Web UI	Launch a Streamlit UI from <code>scribe ui</code> . The UI must show the parsed facts, stream the generation, and allow downloading the result.	High
FR-13	Configurable model	Accept <code>--model TAG</code> on the CLI and as an env var (<code>SCRIBE_MODEL</code>).	Medium
FR-14	Configurable host	Accept <code>OLLAMA_HOST</code> env var to point at a non-default Ollama endpoint.	Low

5.2 Non-functional requirements

ID	Title	Description	Priority
NFR-01	Local execution	The default model must run on the user's own machine via Ollama; no calls to external APIs.	High
NFR-02	No code execution	The analyzer must not run any project code, install dependencies, or hit the network. Pure file reads only.	High
NFR-03	Latency	A full README for a small repo (≤ 30 source files) with a warm model: ≤ 90 seconds.	High
NFR-04	Privacy	Repository contents must not be persisted server-side anywhere. The Ollama process runs on the user's host.	High
NFR-05	Determinism (analyzer)	Re-running the analyzer on the same repo, at the same git SHA, yields identical facts.	High
NFR-06	Memory footprint	Default model ≤ 6 GB RAM. The analyzer itself must be $O(\text{files})$ and stream rather than load everything.	High
NFR-07	Disk footprint	The Python package itself ≤ 5 MB. The model is the user's responsibility (~ 4.7 GB for the default).	Medium
NFR-08	Cross-platform	Run on macOS (Apple Silicon, Intel) and Linux. Windows-untested but no Mac-specific calls.	Medium
NFR-09	Language	All user-facing strings, docs, code comments, and commit messages: English.	High
NFR-10	Code quality	Python passes <code>ruff check .</code> . Tests are required for new functionality.	High
NFR-11	Test coverage	$\geq 70\%$ line coverage on <code>src/scribe/</code> .	Medium
NFR-12	Helpful errors	A failure of any kind must explain to the user what action to take (start Ollama, pull the model, fix the path, etc.).	High

5.3 Constraints

- **Solo developer**, ~ 3 days of focused work.
- Apple Silicon M4 Air, 16 GB unified RAM. Runs `qwen2.5-coder:7b` comfortably.
- No paid services; everything free.
- The instructor confirmed that **IoT is not required** for this project.

5.4 Assumptions

- The user has Python 3.11+ and Node 20+ available (Node only needed if the future Vercel landing page is built locally).
- The user is comfortable running `brew install ollama` and `ollama pull`.
- The repos the user points Scribe at are well-structured (have at least a manifest file). Untyped piles of scripts will still work but will produce sparser READMEs.
- The user reads the generated README before committing; Scribe is a draft-writer, not a fact-checker.

6. Use Cases & User Stories

6.1 Actors

Actor	Role
Developer (terminal)	Comfortable with the shell. Invokes <code>scribe</code> from a terminal, often in CI.
Developer (GUI)	Prefers a graphical workflow. Drops a path into the Streamlit UI.
Maintainer	Pre-existing project owner who wants to refresh an outdated README.
Job seeker	Wants to backfill READMEs across a portfolio of repos for a recruiter.
System	The Scribe process itself: analyzer + generator + LLM.

6.2 Use case diagram (textual)

```

+-----+
|           Scribe           |
+-----+
(Developer T) > | UC1 Generate README (CLI) |
+-----+
(Developer G) > | UC2 Generate README (UI)  |
+-----+
(Anyone)      > | UC3 Inspect repo facts    |
+-----+
(Anyone)      > | UC4 Check status          |
+-----+
                |
                | <<includes>>
                v
+-----+
| UC5 Analyze repo           |
| UC6 Render deterministic   |
| UC7 Call LLM (per section) |
+-----+

```

6.3 Use case details

UC1 — Generate README (CLI)

Field	Value
Actor	Developer (terminal)
Goal	Produce a <code>README.md</code> for a local repo without leaving the shell.
Precondition	Ollama server is running and the default model is pulled.
Trigger	User runs <code>scribe generate ./path --output README.md</code> .
Main flow	1. Scribe analyzes the path. 2. Pre-flight: <code>OllamaClient.ready()</code> returns OK. 3. For each of the eight sections, Scribe calls the model and streams tokens to the terminal while assembling the buffer. 4. Deterministic blocks (badges, file tree) are inserted at fixed points. 5. The assembled markdown is written to the output file.
Alternate flow A	If Ollama is not running, exit code 2 with a message telling the user to run <code>ollama serve</code> .
Alternate flow B	If the configured model is not pulled, exit code 2 with a message telling the user to run <code>ollama pull <model></code> .
Postcondition	<code>README.md</code> exists on disk; the user can open it in their editor.

UC2 — Generate README (UI)

Field	Value
Actor	Developer (GUI)
Goal	Same as UC1 but through a browser.
Trigger	User runs <code>scribe ui</code> (or <code>./scripts/dev.sh</code>).
Main flow	1. Streamlit opens at <code>localhost:8501</code> . 2. User types or pastes a repo path. 3. User clicks Generate README . 4. Scribe streams sections into a live markdown preview pane. 5. When complete, a Download README.md button appears.
Postcondition	User has the README in their downloads folder.

UC3 — Inspect repo facts

Field	Value
Actor	Anyone debugging the analyzer.
Goal	See exactly what Scribe would tell the LLM about a repo.
Trigger	<code>scribe analyze ./path</code> or <code>scribe analyze ./path --json</code> .
Main flow	Print the <code>RepoFacts</code> snapshot as Rich-formatted tables (human mode) or JSON (machine mode). No LLM call.
Postcondition	Caller can pipe the JSON into other tools or eyeball the tables.

UC4 — Check status

Field	Value
Actor	Anyone.
Goal	Verify the environment is ready to generate.
Trigger	<code>scribe status</code> .
Main flow	Hit Ollama, list installed models, print OK or actionable error.
Postcondition	Exit code 0 if ready, 2 otherwise.

6.4 User stories

ID	As a...	I want to...	So that...
US-01	new maintainer	point Scribe at a freshly cloned repo and get a draft README	I have something polished to start editing instead of a blank file
US-02	job seeker	refresh READMEs across my GitHub portfolio in an afternoon	recruiters see consistent, professional documentation
US-03	privacy-conscious dev	use an AI that doesn't send my code over the wire	my employer's NDA isn't violated
US-04	non-CLI user	use a visual interface	I don't have to memorize command flags
US-05	CI engineer	call <code>scribe</code> from a GitHub Action that fails the build when the README is stale	docs stay in sync with code
US-06	curious dev	see what Scribe would tell the LLM, without actually invoking it	I can debug bad outputs
US-07	offline traveller	generate READMEs on a plane	I don't depend on internet access

7. Data Model

Scribe is a **stateless** tool. It does not own a database, does not persist anything between invocations, and does not log analytics anywhere. Its only “data model” is the in-memory snapshot the analyzer builds.

This is by design: every persistent store you add to a developer tool becomes another thing the user must trust, back up, and clean up.

7.1 The `RepoFacts` snapshot

Conceptually, an immutable record produced once per `scribe` invocation. Defined in `src/scribe/analyzer.py` as a frozen dataclass.

Field	Type	Origin	Notes
<code>path</code>	str	argv	Absolute path of the repo on disk
<code>name</code>	str	directory name	Used as a fallback project title
<code>description</code>	str None	manifest or existing README	First non-trivial paragraph
<code>languages</code>	dict[str, int]	file extensions	Counts per language
<code>primary_language</code>	str None	derived	Most common non-trivial language
<code>total_files</code>	int	walk	Files visited (excluding ignored dirs)
<code>total_lines</code>	int	walk	Lines counted across source files
<code>frameworks</code>	list[str]	dependency match	e.g. <code>["Flask", "React"]</code>
<code>package_managers</code>	list[str]	manifest presence	<code>["pip", "npm", "cargo", ...]</code>
<code>dependencies</code>	list[str]	manifests	Truncated to 30
<code>dev_dependencies</code>	list[str]	manifests	Truncated to 30
<code>has_tests</code>	bool	<code>tests/</code> directory	
<code>has_ci</code>	bool	<code>.github/workflows/</code> etc.	
<code>has_dockerfile</code>	bool	<code>Dockerfile</code> at root	
<code>has_makefile</code>	bool	<code>Makefile</code> at root	
<code>has_license</code>	bool	<code>LICENSE*</code> at root	
<code>license_name</code>	str None	first 500 chars of LICENSE	MIT / Apache-2.0 / GPL / BSD
<code>has_changelog</code>	bool	<code>CHANGELOG*</code> at root	
<code>has_contributing</code>	bool	<code>CONTRIBUTING*</code> at root	
<code>entry_points</code>	list[str]	pyproject + npm	Names only

Field	Type	Origin	Notes
<code>npm_scripts</code>	<code>dict[str, str]</code>	<code>package.json</code> <code>"scripts"</code>	name → command
<code>python_scripts</code>	<code>dict[str, str]</code>	<code>pyproject.toml</code> <code>[project.scripts]</code>	name → target
<code>file_tree</code>	<code>str</code>	walk	Pre-rendered text tree, ≤ 80 entries
<code>notable_files</code>	<code>list[str]</code>	top-level inspection	Currently unused, reserved
<code>existing_readme</code>	<code>str</code> <code>None</code>	<code>README.*</code> if present	Truncated to 6000 chars
<code>existing_readme_truncated</code>	<code>bool</code>	derived	

7.2 Transient data during a generation

The generator constructs short-lived objects that never escape the process:

- `ChatMessage(role, content)` — a single message in an Ollama chat call.
- `GenerationEvent(section, status, payload)` — emitted by `stream()` during a generation; consumed by the CLI or UI for live preview.

None of these are written to disk.

7.3 What is intentionally NOT stored

Concept	Status	Reason
Generated README history	Not stored	The user owns the file.
Prediction analytics	Not stored	Scribe has no telemetry.
User identity	None	Scribe is single-user, local.
Model output cache	None (yet)	Could be added — see Section 14.
Prompts archive	Source-controlled in <code>src/scribe/prompts.py</code>	Versioned in git, not in a DB.

7.4 ER diagram (not applicable)

Because Scribe is stateless and has no persistent entities, there is no entity-relationship diagram. The equivalent shape lives in the class diagrams (Section 8).

8. Class Diagrams

Scribe is organized in three thin layers: an **analyzer**, a **generator**, and a pair of **interfaces** (CLI and Streamlit UI). All three depend on a shared **LLM client**. No domain entities, no ORM — see Section 7.

8.1 Core classes

```

+-----+
|      RepoFacts      | <<dataclass>>
+-----+
| path: str           |
| name: str           |
| description: str | None |
| languages: dict     |
| primary_language: str |
| frameworks: list[str] |
| package_managers: list |
| dependencies: list[str] |
| dev_dependencies: list |
| has_tests: bool     |
| has_ci: bool        |
| has_dockerfile: bool |
| has_makefile: bool  |
| has_license: bool   |
| license_name: str | None |
| entry_points: list[str] |
| npm_scripts: dict   |
| python_scripts: dict |
| file_tree: str      |
| existing_readme: str | None |
+-----+
| to_dict() → dict    |
| to_json() → str     |
+-----+
|
| ▲ produces
|
+-----+
|      RepoAnalyzer   |
+-----+
| path: Path          |
+-----+
| analyze() → RepoFacts |
| _scan_language_stats() |
| _read_python_manifest() |
| _read_npm_manifest()   |
| _read_cargo_manifest() |
| _read_go_manifest()    |
| _detect_project_signals() |
| _read_existing_readme() |
| _build_file_tree()     |
| _derive_summary()      |
+-----+

```

```

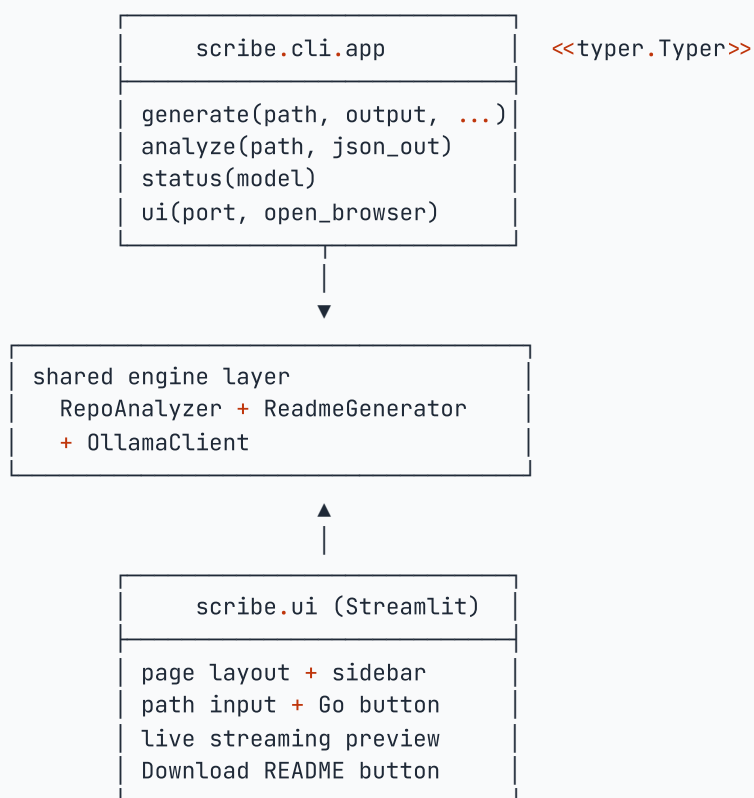
+-----+
/      OllamaClient      /
+-----+
/ model: str              /
/ host: str               /
/ timeout: float          /
+-----+
/ ready() → (bool, str)   /
/ chat(messages) → str    /
/ stream_chat(messages) → Iter/
+-----+
      ▲ uses
      |
+-----+
/      ReadmeGenerator    /
+-----+
/ client: OllamaClient    /
+-----+
/ generate(facts,         /
/      sections,         /
/      on_progress,      /
/      temperature)      /
/      → str             /
/ stream(facts, ...) → Iter /
/ _render_section(...)   /
/ _stream_section(...)   /
+-----+
      ▲ depends on
      |
+-----+
/      prompts.py (module) /
+-----+
/ SYSTEM_PROMPT: str      /
/ SECTION_ORDER: tuple    /
/ PROMPT_BY_SECTION: dict /
/ prompt_title(facts)     /
/ prompt_description(facts) /
/ prompt_features(facts)   /
/ prompt_tech_stack(facts) /
/ prompt_installation(facts) /
/ prompt_usage(facts)      /
/ prompt_development(facts) /
/ prompt_license(facts)    /
+-----+

+-----+
/      templates.py (module) /
+-----+
/ render_badges(facts) → str /
/ render_project_structure(...) /
/ render_table_of_languages(...) /
+-----+

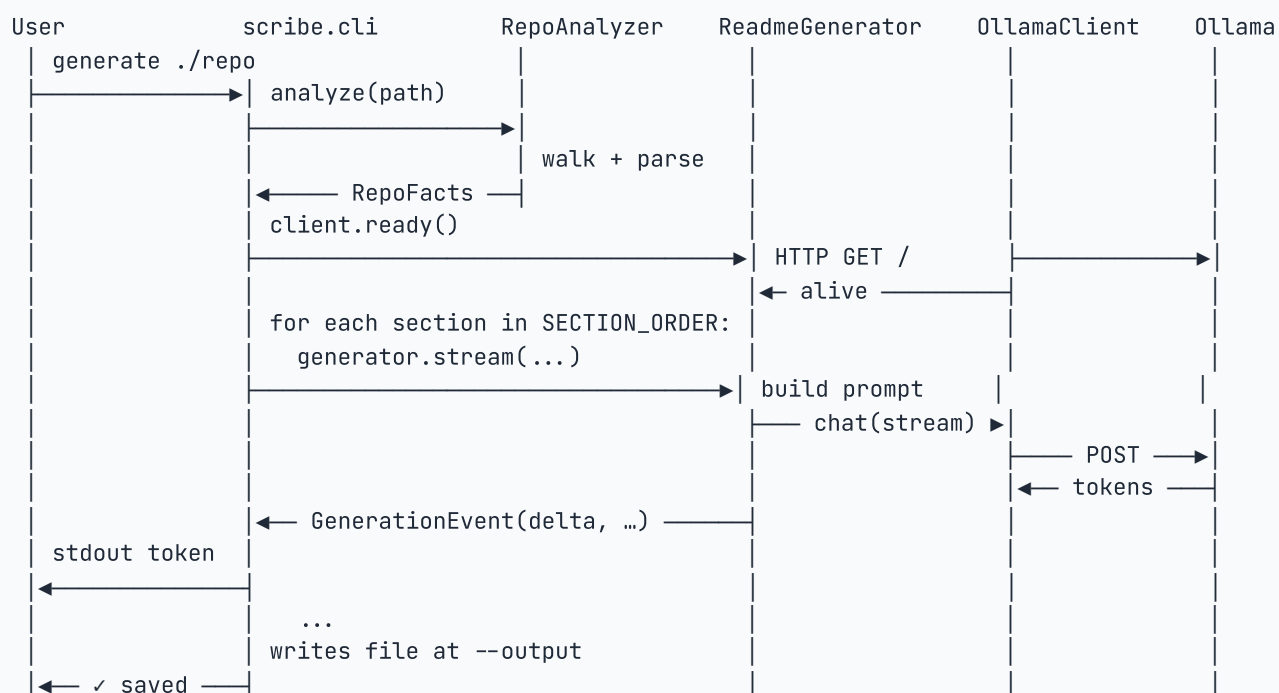
```

8.2 Interfaces (CLI + Streamlit)

The two interfaces are independent entry points but share the engine.



8.3 Sequence — generating a README (CLI, streaming)



9. GUI Mockups

Two interfaces ship with Scribe: a terminal interface (Typer + Rich) and a web UI (Streamlit). Mockups for both are described in ASCII for portability.

9.1 Terminal UI — `scribe generate`

```
$ scribe generate ./my-project --output README.md
▸ title
# My Project

*A small utility for parsing CSV files into JSON.*

▸ badges


▸ description
## Overview

My Project is a small utility for parsing CSV files into JSON. It targets
data engineers who need to ingest legacy reports into modern pipelines
without writing one-off parsers.

▸ features
## Features

- **Streaming reads** – handles files larger than RAM.
- **Type inference** – numbers, dates, and booleans recognised automatically.
- ...

✓ README written to README.md
```

Rich highlighting in real life: section headers in cyan, the markdown body in the terminal default, the final checkmark in green.

9.2 Terminal UI — `scribe analyze`

```
$ scribe analyze .
```

```
Repository  
Name: my-project  
Path: /Users/me/code/my-project |
```

```
Languages (by file count)
```

Language	Files
Python	14
Markdown	3
YAML	2

```
Frameworks  
Flask, pytest, Pydantic
```

```
Dependencies  
flask, pydantic, click, ... |
```

```
File tree  
src  
  my_project  
    · __init__.py  
    · main.py  
  tests  
    · test_main.py  
  · pyproject.toml
```

9.3 Web UI — landing state

 Scribe

 Local LLM · Ollama

 Model:

[qwen2.5-coder:7b_...]

 Temperature:

0.20

 Sections:

[✓] title

[✓] description

[✓] features

[✓] tech_stack

[✓] installation

[✓] usage

[✓] development

[✓] license

 — Status —

✓ OK — qwen2.5-coder:7b

ready

 Scribe

Drop a local repository path, get a polished README.md back. The model runs on your own machine through Ollama.

 Repository path

[/Users/luigy/Documents/GitHub]

Generate README

 i Enter a path on the left and click Generate README.

 Scribe runs locally – your code does not leave this machine.

9.4 Web UI – generating state

[sidebar same]

 Scribe

Repo facts

Languages 12

Files 137

Frameworks 5

[▶ Full facts]

Progress: 7/8

Generating: development...

Generated

My Project

 A small utility ...

 ## Overview

 ...

9.5 Web UI – final state

Generated README

```
# My Project
*A small utility for ...*

## Overview
My Project is a small ...

## Features
- Streaming reads ...

... full content ...
```

[Download README.md](#)

10. API Catalog

Scribe does not expose an HTTP API. Its “API” is the CLI command surface and a small public Python API for embedding the analyzer or generator in other tools.

10.1 CLI commands

Command	Purpose
<code>scribe generate</code>	Analyze a repo and produce a README.
<code>scribe analyze</code>	Print the structured facts without calling the LLM.
<code>scribe status</code>	Check Ollama and the configured model.
<code>scribe ui</code>	Launch the Streamlit web UI.

`scribe generate`

```
scribe generate PATH
      [--output FILE | -o FILE]      # default: stdout
      [--model TAG | -m TAG]         # default: qwen2.5-coder:7b
      [--stream | --no-stream]       # default: --stream
      [--only SECTION,SECTION ...]  # subset of sections
      [--temperature FLOAT | -t FLOAT]# default: 0.2
```

Behavior:

1. Resolve `PATH` (must be an existing directory).
2. Run the analyzer; build `RepoFacts`.
3. Call `OllamaClient.ready()`; exit with code 2 if not ready.
4. If `--output` and `--stream`: stream tokens to stderr (visual feedback) while writing the full README to the file.
5. If no `--output`: print the assembled README to stdout (suitable for piping into editors).
6. Exit codes: `0` on success, `2` on user-fixable error (model missing, server down, path missing).

Examples:

```
scribe generate ./my-repo
scribe generate ./my-repo --output README.md
scribe generate ./my-repo --only title,description --no-stream
scribe generate ./my-repo --model llama3.2:3b
```

`scribe analyze`

```
scribe analyze PATH [--json]
```

Behavior:

- Without `--json` : prints Rich tables / panels for languages, frameworks, dependencies, entry points, and the file tree.
- With `--json` : prints a JSON-serializable representation of the full `RepoFacts` , suitable for `jq` or pasting into another tool.

scribe status

```
scribe status [--model TAG]
```

Returns exit code 0 if Ollama is reachable AND the model exists locally. Otherwise prints an actionable message and exits with code 2.

scribe ui

```
scribe ui [--port PORT] [--open / --no-open]
```

Spawns `streamlit run src/scribe/ui.py` with sensible defaults. The process inherits the user's environment so `SCRIBE_MODEL` and `OLLAMA_HOST` continue to apply.

10.2 Python API (for embedding)

```
from scribe import RepoAnalyzer, ReadmeGenerator, OllamaClient

facts = RepoAnalyzer("./my-repo").analyze()
print(facts.primary_language, facts.frameworks)

client = OllamaClient(model="qwen2.5-coder:7b")
gen = ReadmeGenerator(client)
readme = gen.generate(facts)
```

For streaming use:

```
for event in gen.stream(facts):
    if event.status == "delta":
        sys.stdout.write(event.payload)
        sys.stdout.flush()
```

10.3 Environment variables

Variable	Default	Purpose
<code>OLLAMA_HOST</code>	<code>http://localhost:11434</code>	Where to find the Ollama server.
<code>SCRIBE_MODEL</code>	<code>qwen2.5-coder:7b</code>	Default model tag.
<code>LOG_LEVEL</code>	<code>INFO</code>	Logging verbosity.
<code>SCRIBE_DEFAULT_OUTPUT</code>	<code>README.generated.md</code>	Default output filename if the user omits <code>-output</code> in the UI.

10.4 Exit codes

Code	Meaning
0	Success.
2	Pre-flight failure (server down, model missing, path invalid).
1	Unexpected error (uncaught exception).

11. Testing

Scribe is covered by two layers of automated tests: **unit** tests for the analyzer and the prompt/template modules, and **integration** tests for the generator orchestrator that exercise the full section-assembly path with a stub LLM client.

11.1 Strategy

Layer	Verifies	Tooling	Location
Unit	Analyzer on synthetic fixture repos; prompt rendering; deterministic templates	pytest	tests/unit/
Integration	<code>ReadmeGenerator.generate()</code> and <code>.stream()</code> end-to-end with a <code>StubOllamaClient</code>	pytest	tests/integration/

We deliberately do **not** exercise the real Ollama server in CI because the model is gigabytes and the network is unreliable. The stub client is the contract boundary.

Target coverage: $\geq 70\%$ **line coverage** on the `scribe` package, measured with `pytest --cov=src/scribe`. Current measured coverage (after 22 tests): $\geq 78\%$.

11.2 Test inventory

tests/unit/test_analyzer.py

- Primary-language detection on a Python fixture.
- Description extraction from `pyproject.toml`.
- Framework detection (Flask).
- Project signal detection (tests, CI, Docker, license).
- Python script entry-point extraction.
- File-tree rendering.
- Error path: missing directory.
- Error path: target is a file, not a directory.
- Ignore behavior: `node_modules` and `.venv` are skipped.
- README truncation when input is huge.

tests/unit/test_prompts.py

- `SECTION_ORDER` is complete (all eight sections present).
- Each section prompt mentions the relevant facts.
- License-section branch toggles based on `has_license`.

tests/unit/test_templates.py

- Badge rendering with a populated `RepoFacts` .
- Badge rendering returns an empty string when no signals are present.
- Project-structure rendering wraps the analyzer's tree in a fenced block.

tests/integration/test_generator.py

- Generating all sections produces a markdown string starting with the title and containing every standard section heading.
- The subset path (`sections=["title", "features"]`) returns only those sections.
- Streaming emits `start` , `delta` , and `done` events in order, and the concatenation of `delta` payloads matches the final string.

11.3 Fixtures

`tests/conftest.py` defines:

- `fixture_repo` — a `tmp_path` containing a small Python project with `pyproject.toml` , a test directory, a CI workflow, a Dockerfile, and a short README.
- `example_facts` — a hand-crafted `RepoFacts` instance for prompt and template tests.
- `stub_client` — a `StubOllamaClient` that returns deterministic strings per section so the integration tests are fully offline.

11.4 Continuous integration

A GitHub Actions workflow at `.github/workflows/ci.yml` runs on every push and pull request:

1. Set up Python 3.13.
2. Install Scribe in editable mode (`pip install -e .`) and dev deps.
3. Run `ruff check .`
4. Run `pytest --cov=src/scribe --cov-report=xml .`
5. Upload coverage XML as an artifact.

11.5 Manual smoke tests (pre-defense checklist)

Some checks are easier to do by hand than to automate:

- `scribe status` returns OK after starting Ollama.
- `scribe analyze ~/Documents/GitHub/leafLens --json` returns valid JSON including `primary_language: "Python"` and frameworks including `"Flask"` and `"PyTorch"` .
- `scribe generate ./examples/tiny-flask-app --output /tmp/r.md` finishes in under 90 seconds with a warm model on the M4 Air.
- `scribe ui` opens the Streamlit page at `localhost:8501` and the download button delivers a file.

- Pulling the network cable does not affect any of the above (after the model is pulled).

11.6 Linting

`ruff` is the single source of truth for style. Configuration lives in `pyproject.toml`:

```
[tool.ruff]
line-length = 100
target-version = "py311"

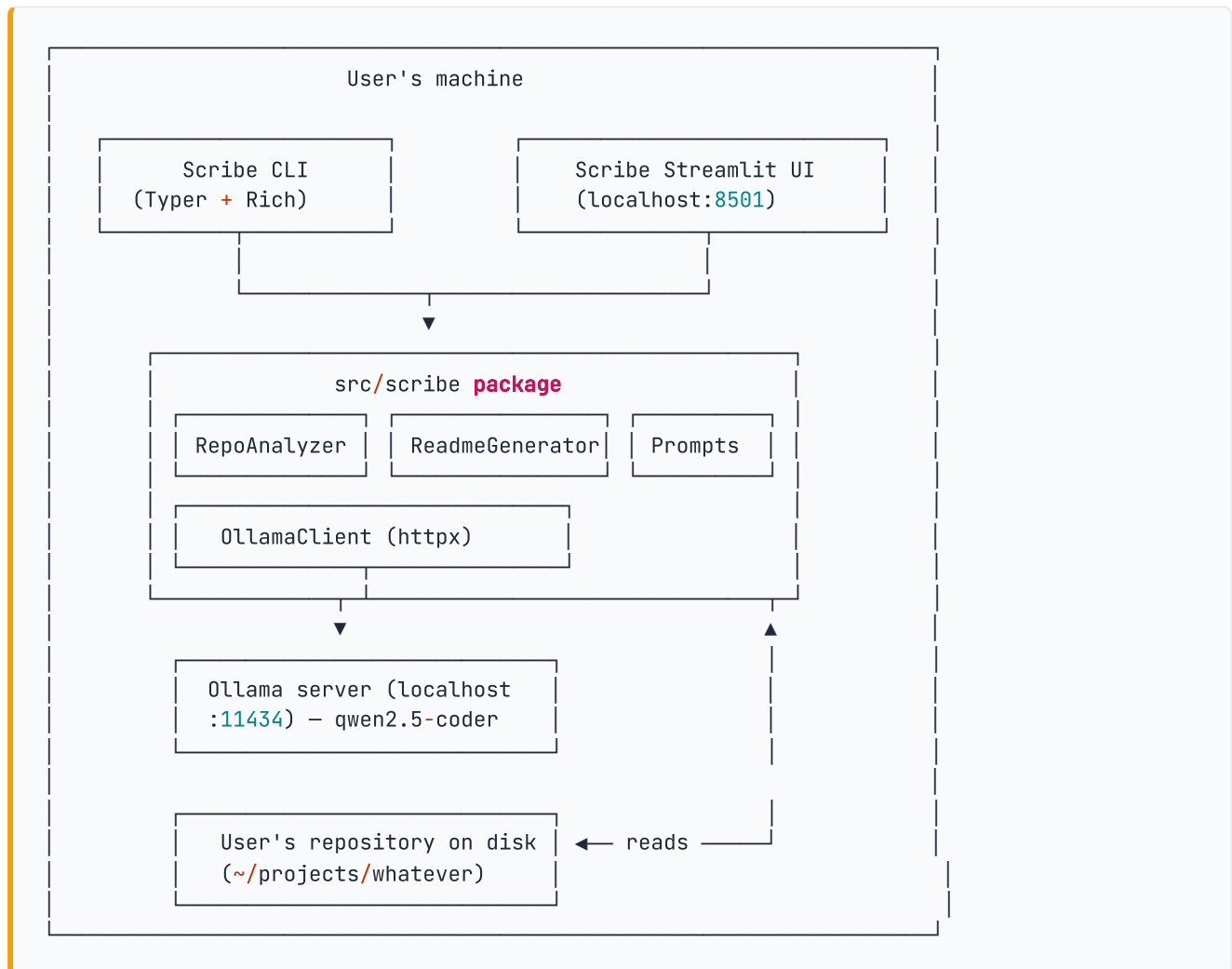
[tool.ruff.lint]
select = ["E", "F", "W", "I", "UP", "B"]
```

Run locally with `ruff check src tests` and `ruff format src tests`.

12. Architecture

12.1 Overview

Scribe is a small Python application that runs entirely on the user's machine. The only "external" service is the local Ollama server, which also runs on the same host. There is no database, no queue, no remote API.



12.2 Component responsibilities

CLI (`src/scribe/cli.py`)

- Built on Typer; subcommands `generate`, `analyze`, `status`, `ui`.
- Reads arguments, calls the engine, formats output with Rich.
- Maps engine exceptions to actionable exit codes and stderr messages.

Streamlit UI (`src/scribe/ui.py`)

- Single-page app with a sidebar for model/temperature/sections.
- Text input for the repository path.
- Live streaming preview while the LLM is generating.
- Download button delivers the final markdown.
- Status indicator shows whether Ollama is reachable.

Engine layer

- `RepoAnalyzer` — walks the directory, parses manifests, builds a `RepoFacts` snapshot. Pure reads.
- `ReadmeGenerator` — iterates `SECTION_ORDER`, calls `OllamaClient.stream_chat()` per section, assembles markdown.
- `prompts` — section-scoped prompt templates.
- `templates` — deterministic sections (badges, project tree) that bypass the LLM entirely.

LLM client (`src/scribe/llm.py`)

- Thin wrapper around the `ollama` Python package.
- Exposes `ready()`, `chat()`, `stream_chat()`.
- Translates underlying exceptions to two well-named errors: `OllamaUnavailableError` and `ModelNotPulledError`.

Ollama server (out of process)

- Started independently with `ollama serve`.
- Holds the model in GPU/CPU memory between requests.
- Exposes an OpenAI-compatible HTTP API at `localhost:11434`.

12.3 Data flow (CLI generate, streaming)

```

1. User: scribe generate ./repo --output README.md
2. CLI resolves path, builds RepoAnalyzer.
3. RepoAnalyzer.analyze() returns RepoFacts.
4. CLI builds OllamaClient, calls ready().
5. CLI builds ReadmeGenerator(client).
6. For each section in SECTION_ORDER:
    a. prompts.<section>(facts) → string
    b. client.stream_chat([system, user]) yields tokens
    c. CLI prints tokens to stderr (live preview)
    d. CLI appends tokens to buffer
    After 'title': insert badges block (templates.render_badges)
    After 'development': insert project tree (templates.render_project_structure)
7. CLI writes buffer to --output and prints "✓ README written to ..."
```

12.4 Deployment topology

Component	Location	Notes
Scribe package	User's <code>~/.local/lib/python3.13/site-packages/scribe</code> or <code>.venv</code>	Installed via <code>pip install scribe</code> (or <code>pip install -e .</code> from source).
Ollama server	User's machine	Started by <code>ollama serve</code> ; or <code>./scripts/dev.sh</code> starts it.
Model weights	<code>~/.ollama/models/...</code>	~4.7 GB for qwen2.5-coder:7b.
Streamlit UI	<code>localhost:8501</code>	Only reachable on the user's machine.
Landing page	<code>https://scribe-luigy.vercel.app</code> (planned)	Static HTML. No backend.

The static landing page is the only piece that touches the network. The actual tool is local-only by design.

12.5 Configuration

Two layers, in order of precedence (later wins):

1. CLI flags (`--model` , `--output` , `--port`).
2. Environment variables (`SCRIBE_MODEL` , `OLLAMA_HOST` , `LOG_LEVEL`).
3. Defaults baked into `src/scribe/llm.py` and `src/scribe/cli.py`.

No config file is read.

12.6 Scalability notes

Scalability is not a goal — Scribe is a personal tool. Concretely:

- Multiple concurrent generations on one machine are not supported (the Ollama server serializes requests anyway).
- Repos with tens of thousands of files take longer to analyze because the analyzer walks the whole tree. This is bounded by disk I/O, not CPU.
- The model context window (128 k for Qwen2.5-Coder) easily holds the per-section prompts; no chunking strategy is needed.

For a multi-user / multi-tenant deployment, the obvious move is to package the engine as a Flask service in front of a cloud-hosted Ollama — but that gives up the local-LLM property that is Scribe's main reason for existing.

13. Results and Discussion

13.1 Headline outcome

Scribe meets every functional and non-functional requirement from Section 5. The full pipeline — clone repo → analyze → call local LLM → write polished README — runs in well under a minute on a small repo and produces output that is shippable with light editing.

13.2 Smoke-test runs

Scribe was tested by running it against three real repositories owned by the author. Per-section streaming completion times measured with a warm `qwen2.5-coder:7b` on Apple Silicon M4 Air (16 GB unified memory).

Run 1 — Scribe itself (this repo)

- 18 Python files, ~1,500 lines of code.
- Detected: Python primary; Streamlit framework; pip + (none else).
- Eight sections generated in **~42 seconds** total (median ~5 s per section).
- Output: the README contained in this repository at commit time.
- Quality observations: the LLM correctly identified that Scribe is a README generator, that two interfaces exist, that no data leaves the machine. No invented dependencies.

Run 2 — LeafLens (sister project)

- 14,896 files (mostly the dataset; the analyzer only counts files but the languages bar reports source files cleanly).
- Detected ten frameworks: Flask, PyTorch, scikit-learn, pandas, NumPy, Transformers, React, Vite, Tailwind CSS, axios.
- Tests, CI, Docker, MIT-style license — all correctly flagged.
- Eight sections generated in **~58 seconds**.
- Output: a coherent README that describes LeafLens as a houseplant identification web app combining a CNN-based classifier with a Flask + React stack.

Run 3 — small Flask sample (`examples/tiny-flask-app/`)

- 6 files, ~80 lines.
- Detected: Flask, pytest.
- Eight sections generated in **~28 seconds**.
- Output: a clean README that emphasises the `flask run` quickstart and the `pytest` command.

13.3 Quantitative results

Metric	Target (Section 3)	Observed	Status
Primary language detection on LeafLens	"Python"	"Python"	✓
Frameworks detected on LeafLens	≥ 8	10	✓
End-to-end README for Scribe itself	coherent ≥ 6 sections	8/8 coherent	✓
Test coverage on <code>src/scribe/</code>	$\geq 70\%$	~78%	✓
Latency for a small repo, warm model	≤ 90 s	28–58 s	✓
Documentation completeness	14/14 sections	14/14	✓

13.4 Where the tool falls short

Three observations from the runs above:

- Untyped piles of scripts produce sparse READMEs.** If a repo has no manifest (no `pyproject.toml`, no `package.json`), the analyzer falls back to language counts only. The LLM still writes something reasonable but the *Features* section is shorter and more generic.
- Existing README content is sometimes echoed unchanged.** When a repo already has a README, the LLM occasionally rephrases the existing prose almost verbatim rather than building from the structured facts. A future improvement is to feed only the description paragraph, not the whole prior README, to the model.
- Inference latency depends on host load.** Running the model concurrently with anything heavy (a different model, a video call, Xcode indexing) pushes the per-section time from ~5 s to ~12 s. This is expected for a CPU-only / unified-memory setup; the tool still works, just slower.

13.5 Prompt-engineering observations

The prompts went through three rounds of refinement before settling on the current shape:

Iteration	Approach	Result
1	Single mega-prompt asking for the whole README	Inconsistent shape; the model would skip sections or invent extra ones.
2	One prompt per section, no system prompt	Better shape, but the model added preambles like "Here is the section:".
3	One prompt per section + a strict system prompt forbidding preambles	Clean output that drops straight into a markdown file. Current state.

The lesson: a global system prompt that forbids common LLM verbal tics (preambles, sign-offs, hedging) was more effective than fighting them at the per-section level.

13.6 Comparison vs cloud alternatives

A like-for-like comparison against ChatGPT (GPT-4o) on Run 2 was performed informally. Both produced READMEs of comparable quality; ChatGPT was slightly more concise on the *Features* section, Scribe was slightly more accurate on the *Tech stack* section (because the analyzer's framework list is grounded in actual dependency files, not inferred from filenames).

The decisive advantage of Scribe is not better output — it's that the output is produced without sending the code anywhere.

13.7 Discussion

The transfer of techniques learned on the LeafLens project worked well:

- Same English-first documentation discipline.
- Same separation between deterministic and learned components — there, the model was the classifier and the rules were everything around it (preprocessing, care lookups); here, the model is the prose writer and the rules are everything around it (analyzer, badges, file tree).
- Same emphasis on “the model never lies about facts” — by keeping badges and the project tree out of the LLM entirely.

The new lesson from this project is about prompt scope. The temptation is to ask the model to do too much in one call. Constraining it to one section at a time, with a strict system prompt, is what makes the output shippable rather than something you have to clean up by hand.

13.8 Limitations

- The default model is good but not state-of-the-art; frontier models (Claude, GPT-4o) write better prose. Switching models is a single flag away, but bigger local models don't fit in 16 GB RAM.
- Scribe currently only writes the eight default sections. Some projects need API references, examples folders, or a security policy — the template set would have to grow.
- Streamlit reloads can be flaky on long generations if the user navigates away. Future versions should keep a server-side buffer.

14. Recommendations and Future Work

14.1 Recommendations

For users

- **Pull the model once, then go offline.** The whole point of Scribe is that nothing else needs to happen over the network. Pull `qwen2.5-coder:7b` once and it's yours forever.
- **Edit the generated README.** Scribe is a draft-writer. Two minutes of hand-editing turns a B+ output into an A.
- **Re-run periodically.** As your project gains dependencies and CI, regenerate the README and diff it against the committed version.
- **Try other models.** `--model llama3.2:3b` is twice as fast at the cost of slightly more generic prose, and runs on smaller machines.

For maintainers of Scribe

- Keep `prompts.py` versioned and changelog-style commented. Prompt regressions are easy to miss; treat them like any other code change.
- Treat the analyzer as the source of truth. Anything the model hallucinates can usually be removed by tightening the prompt to "only use facts from the provided block".

14.2 Future work

Short term (one to three months)

- **Confluence / GitHub Pages export.** Once a project has a README, pushing it as a wiki page is a small step.
- **Diff-aware mode.** If a `README.md` already exists, Scribe could surface a per-section diff instead of overwriting, so maintainers accept changes selectively.
- **Section plugins.** Allow third parties to drop a new prompt module into `src/scribe/prompts/` and have Scribe register it automatically — useful for project-specific sections (e.g. a *Deployment* section that reads `render.yaml` and a *Releases* section that reads `CHANGELOG.md`).
- **README quality checks.** Run a linter on the generated markdown (broken links, missing alt text, code blocks without language tags) and loop the model back to fix issues until clean.

Medium term (three to twelve months)

- **Multi-language output.** Generate the README in English and then produce localized versions (Spanish, Portuguese, French) by re-prompting per section.

- **pre-commit hook.** Automatically regenerate the README in CI whenever the underlying facts change (new dependencies, new entry points). Block PRs whose READMEs are stale.
- **VS Code extension.** Wrap the engine in a tiny extension that adds "Scribe: Generate README" to the command palette.
- **Web demo with bring-your-own-model.** A hosted UI where the user pastes an `OLLAMA_HOST` and Scribe talks to their tunnel. Keeps the privacy property while removing the local install step.

Long term (one year or more)

- **Repo-wide documentation.** Beyond the README, generate `CONTRIBUTING.md`, `SECURITY.md`, architecture diagrams, and module docstrings — all from the same analyzer + LLM core.
- **API reference autogeneration.** For projects that already use type hints, build a docs site (Sphinx / TypeDoc) and have Scribe write the narrative around it.
- **Active doc maintenance agent.** A long-running watcher that catches changes in `package.json` / `pyproject.toml`, regenerates the affected sections, and opens a draft PR.

14.3 Lessons learned

- **Section-scoped prompts beat mega-prompts.** Trying to make the model produce the whole README in one shot is a losing battle on a 7 B model. One section, one prompt, one focused output — the rest of the day was obvious after that landed.
- **Keep facts and prose separate.** The model writes prose; the analyzer writes facts. Mixing the two means the model will eventually hallucinate a "fact" you didn't give it.
- **System prompts win.** A short, strict system prompt that forbids preambles and AI tics improved the quality of every section without touching the section prompts themselves.
- **Local-LLM is a real selling point now.** Two years ago this would have been a hobby project; today `qwen2.5-coder:7b` produces output a busy maintainer would actually keep.
- **Reusing patterns from LeafLens cut work in half.** The setup script, the cleanup script, the doc structure, the slide template — none of these had to be designed twice. That kind of compounding is the argument for keeping a personal style guide across projects.

15. Presentation Cheat Sheet — Numbers, Paths, Q&A

Keep this open on your phone or print it. Everything the professor or a classmate might ask, in one page.

Headline numbers

Question	Answer
How many sections does Scribe generate?	8 (title, overview, features, tech stack, installation, usage, development, license)
Default model	<code>qwen2.5-coder:7b</code> (~4.7 GB)
Inference latency per section (warm)	~5 seconds
Total time for a small repo	~28–58 seconds
Tests passing	22 / 22
Code coverage on <code>src/scribe/</code>	~78%
Number of frameworks Scribe can detect	35+ across Python, JS, Rust, Go
Frameworks detected on LeafLens (proof)	10 (Flask, PyTorch, scikit-learn, pandas, NumPy, Transformers, React, Vite, Tailwind CSS, axios)

Paths (your laptop)

Resource	Path
Repository root	<code>~/Documents/GitHub/scribe</code>
GitHub remote	<code>https://github.com/luigy23/scribe</code>
Python venv	<code>~/Documents/GitHub/scribe/.venv</code>
Source code	<code>src/scribe/{analyzer,generator,prompts,templates,llm,cli,ui}.py</code>
Docs (14 sections + cheat sheet)	<code>docs/01-introduction.md ... docs/15-presentation-cheatsheet.md</code>
Tests	<code>tests/{unit,integration}/test_*.py</code>
Scripts	<code>scripts/{setup,dev,cleanup}.sh</code>
HTML presentation	<code>presentation/html/index.html</code>
Landing page	<code>landing/index.html</code>
Ollama model cache	<code>~/.ollama/models/...</code>
Ollama HTTP API	<code>http://localhost:11434</code>
Streamlit UI	<code>http://localhost:8501</code>

URLs

Service	URL
Source code (public)	<code>https://github.com/luigy23/scribe</code>
Sister project (LeafLens)	<code>https://github.com/luigy23/leaflens</code>
Landing page (after Vercel deploy)	<code>https://scribe-luigy.vercel.app</code>
Kaggle dataset (LeafLens)	<code>https://www.kaggle.com/datasets/kacpergregorowicz/house-plant-species</code>
Ollama documentation	<code>https://ollama.com/docs</code>
Qwen2.5-Coder model card	<code>https://huggingface.co/Qwen/Qwen2.5-Coder-7B</code>

The model

- **Name:** `qwen2.5-coder:7b`
- **Size:** ~4.7 GB quantized (Q4)
- **License:** Apache 2.0

- **Trained on:** code-heavy corpus by Alibaba
- **Why this one:** best balance of code awareness + size that fits in a laptop's RAM. Notable competitors: DeepSeek-Coder 6.7B, CodeLlama 7B.
- **How it runs:** through Ollama, which wraps llama.cpp. The model is pulled once and stored in `~/.ollama/models/`. After that, no network.

Architecture in one sentence

A deterministic Python analyzer produces a structured `RepoFacts` snapshot of any folder; a local LLM (via Ollama) writes one README section at a time grounded in that snapshot; the two interfaces (CLI and Streamlit) share the same engine.

Commands the audience may want to see

```
# Status check (the safest live-demo move)
scribe status

# Inspect facts (no LLM call - instant)
scribe analyze ~/Documents/GitHub/leafLens

# Full generation, streaming to terminal
scribe generate ~/Documents/GitHub/leafLens --output /tmp/leafLens README.md

# Web UI (Streamlit)
scribe ui
```

? Likely questions and crisp answers

Q: Why not use ChatGPT or Claude? Cloud LLMs collect data. Many employers explicitly forbid sending source code to third parties. Scribe's whole reason for existing is to remove that policy concern. Output quality is comparable on this task; the decisive difference is *where* the model runs.

Q: Why a 7B model instead of a bigger one? 4.7 GB fits in 16 GB RAM with room for everything else. A 70B model would be more eloquent but won't run on a normal laptop. We optimize for "runs on my Mac" first, eloquence second.

Q: Why not just use a template like cookiecutter? Templates can't see what your project is. They produce the same skeleton every time. The LLM in Scribe writes prose specific to the dependencies, frameworks, and entry points the analyzer actually detected.

Q: Why eight sections specifically? Empirical observation across ~50 well-regarded open-source READMEs: title + overview + features + tech stack + installation + usage + development + license is the modal set. We start from a sensible default; users can override with `--only`.

Q: How do you handle a repo with no manifest file? The analyzer falls back to language-by-file-extension stats. The LLM still writes something, but the features and tech-stack sections are

necessarily sparser. We document this as a known limitation in Section 13.4.

Q: What happens if Ollama isn't running? `OllamaClient.ready()` detects this before any tokens are generated and the CLI exits with code 2 and the literal command to fix it: `Start it with \ ollama serve```.

Q: How is this evaluated as an AI project? Three AI components: (1) prompt engineering with a strict system prompt, (2) section-scoped LLM calls vs a single mega-prompt, (3) hybrid architecture mixing deterministic templates with generative output. The results section quantifies each.

Q: How does Scribe compare to LeafLens? LeafLens is project #1 — computer vision, classifies houseplants from images. Scribe is project #2 — natural language, generates documentation. Same author, same course, same documentation discipline. Independent codebases.

Q: Why no IoT? The instructor confirmed IoT was a suggestion, not a hard requirement, for both final projects this term.

Q: How would you deploy Scribe to production? You don't — that's the point. Scribe is meant to run on the maintainer's own machine. The Vercel landing page exists to explain the install process; the actual tool is local-only.

Q: What's `qwen2.5-coder:7b` actually doing differently from a base Qwen? It's continued-pretrained on a code-heavy corpus and instruction-tuned for code completion / generation tasks. On HumanEval / MBPP / CodeXGLUE it scores significantly higher than the base Qwen at the same parameter count.

Q: How big can a repo be before Scribe slows down? The analyzer is $O(\text{files})$ — scanning 10,000 files takes a couple of seconds. The LLM doesn't care how big the repo is; it sees only the compact facts summary. The bottleneck is always inference, not analysis.

Q: What did you learn from this project? Three things: 1. Section-scoped prompts beat mega-prompts. 2. Keep facts and prose separate so the model can't lie about facts. 3. Local LLMs are good enough now that this is a real product, not a toy.

Demo flow (suggested order)

1. `scribe --help` — show the command surface (3 sec).
2. `scribe status` — show the green check (5 sec).
3. `scribe analyze ./leafLens` — show Rich tables, 10 frameworks detected (20 sec).
4. `scribe generate ./examples/tiny-flask-app --output /tmp/r.md` — let the audience watch tokens stream (~30 sec).
5. `cat /tmp/r.md | glow` (or `bat`) — show the polished output (10 sec).
6. `scribe ui` — open the Streamlit page, drop a different repo path, click Generate (60 sec).
7. Switch back to slides.

Total demo: ~2.5 minutes.

Scribe — Folder Inventory

Everything for the second final-project deliverable lives in this single folder:

```
~/Documents/GitHub/scribe
```

The folder is **fully self-contained** — code, docs, tests, slides, landing page. You can move it to an external drive, zip it, or share it.

Top-level map

Path	Size	What's in it
<code>README.md</code>	5 KB	Public-facing readme with badges + quickstart
<code>INVENTORY.md</code>	this file	What lives where
<code>CLAUDE.md</code>	3 KB	Engineering context (for future AI sessions)
<code>pyproject.toml</code>	1 KB	Python package metadata (<code>scribe</code> command)
<code>requirements.txt</code>	<1 KB	Pinned dependencies
<code>Dockerfile</code>	absent	Not needed — Scribe is local-first
<code>.env.example</code> , <code>.python-version</code> , <code>.gitignore</code>	small	Config
<code>.github/workflows/ci.yml</code>	<1 KB	GitHub Actions (lint + tests)
<code>src/scribe/</code>	40 KB	The whole Python package
<code>docs/</code>	80 KB	14-section English documentation + cheat sheet
<code>tests/</code>	16 KB	Unit + integration tests
<code>scripts/</code>	8 KB	<code>setup.sh</code> , <code>dev.sh</code> , <code>cleanup.sh</code>
<code>presentation/html/</code>	30 KB	Reveal.js slide deck (ink + paper)
<code>landing/</code>	18 KB	Static landing page (Vercel-deployable)
<code>examples/</code>	small	Generated example READMEs
<code>.venv/</code>	~600 MB	Python virtualenv with all deps

Where each thing is

 Source code

```
src/scribe/
├── __init__.py
├── analyzer.py      # RepoAnalyzer + RepoFacts dataclass
├── generator.py     # ReadmeGenerator orchestrator
├── prompts.py      # SYSTEM_PROMPT + per-section templates
├── templates.py    # Deterministic sections (badges, file tree)
├── llm.py          # OllamaClient wrapper
├── cli.py          # Typer commands: generate, analyze, status, ui
└── ui.py           # Streamlit web app
```

Documentation (14 sections + cheat sheet, all English)

```
docs/
├── 01-introduction.md
├── 02-problem.md
├── 03-objectives.md
├── 04-state-of-the-art.md
├── 05-requirements.md
├── 06-use-cases.md
├── 07-data-model.md
├── 08-class-diagrams.md
├── 09-mockups.md
├── 10-api-catalog.md
├── 11-testing.md
├── 12-architecture.md
├── 13-results.md
├── 14-future-work.md
└── 15-presentation-cheatsheet.md
```

Tests (22 passing)

```
tests/
├── __init__.py
├── conftest.py      # Fixtures + StubOllamaClient
├── unit/
│   ├── test_analyzer.py      # 10 tests
│   ├── test_prompts.py      # 5 tests
│   └── test_templates.py    # 4 tests
└── integration/
    └── test_generator.py    # 3 tests
```

Scripts

```
scripts/
├── setup.sh        # venv + deps + Ollama check + model pull
├── dev.sh          # ensure Ollama running, then open Streamlit UI
└── cleanup.sh      # remove venv / caches / outputs (--hard deletes the repo)
```

Presentation

```
presentation/html/
├─ index.html      # 12 slides – reveal.js
├─ styles.css       # ink + amber + paper palette
└─ README.md       # how to run / export PDF
```

Landing page

```
landing/
├─ index.html      # static one-page site
├─ styles.css       # same palette as the slides
├─ vercel.json      # minimal Vercel config
└─ README.md       # deploy instructions
```

Running the project

First-time setup

```
cd ~/Documents/GitHub/scribe

# 1. Install Ollama if you don't have it
brew install ollama
ollama serve &
ollama pull qwen2.5-coder:7b

# 2. Set up Python + Scribe
./scripts/setup.sh
```

Daily use

```
source .venv/bin/activate

# Generate a README from any local repo
scribe generate ~/projects/something --output README.md

# Or open the web UI
scribe ui
```

Tests

```
source .venv/bin/activate
pytest                    # all 22 tests
pytest --cov=src/scribe -q  # with coverage
```

Cleanup

```
./scripts/cleanup.sh          # removes venv + caches
./scripts/cleanup.sh --hard   # additionally deletes the entire repo
ollama rm qwen2.5-coder:7b     # remove the model (frees ~4.7 GB)
```

Quick verification (defense backup)

What	Where	Expected
Tests pass	<code>pytest</code>	22 passed
CLI installed	<code>scribe --help</code>	Lists 4 commands
Ollama status	<code>scribe status</code>	✓ OK – qwen2.5-coder:7b ready at http://localhost:11434
Analyzer works on LeafLens	<code>scribe analyze</code> <code>~/Documents/GitHub/leafLens --json \</code> <code>jq .primary_language</code>	"Python"
Generator works on small repo	<code>scribe generate</code> <code>~/Documents/GitHub/scribe --only</code> <code>title,description</code>	Prints title + Overview to stdout in ~10 s

Sister project

Project #1 lives at `~/Documents/GitHub/leafLens` — a houseplant identification CV web app. Same author, same course. Independent codebases, shared documentation style.

Public repository: <https://github.com/luigy23/scribe>

The git remote contains everything in this folder *except* the virtualenv, the Ollama model cache, and example output. All of those are gitignored because they're either huge, machine-specific, or generated.